
Major Assignment 1 and 2- CS771

Abhishek L
230049

Soubhagya Raha
231025

Subhayan Dan
231043

Mainak Sarkar
230619

Ayush Raj
230267

Bharat Kumar
230289

Problem 1.1: Melbo is Hiring

1. Derivation of the Semi-Parametric Kernel

The objective is to model the number of staff y required for a video, given its length x and topic-related properties $z = [z_1, z_2]$, which represent the topic's popularity and difficulty. It is reasonable to assume that, for a fixed topic z , the staff requirement grows approximately linearly with the video length x . However, the proportionality constant of this relationship, denoted $w(z)$, is not fixed — it varies with the topic itself. This gives the semi-parametric form:

$$y = w(z)x + b,$$

where b is a bias term representing a baseline staffing requirement independent of x .

The dependence of $w(z)$ on z is nonlinear, so we model it as:

$$w(z) = p^\top \phi(z),$$

where $\phi(z)$ maps the two-dimensional topic vector z into a higher-dimensional feature space that captures nonlinear relationships, and p is a parameter vector in that feature space. Substituting this back gives:

$$y = p^\top (x \phi(z)) + b.$$

To make this model compatible with kernel-based methods, we define an **augmented feature map**:

$$\psi(x, z) = \begin{bmatrix} x \phi(z) \\ 1 \end{bmatrix}, \quad \tilde{p} = \begin{bmatrix} p \\ b \end{bmatrix}.$$

The model can now be written linearly in the new feature space:

$$y = \tilde{p}^\top \psi(x, z).$$

Thus, a natural kernel between two data points (x, z) and (x', z') is defined as:

$$\tilde{K}((x, z), (x', z')) = \langle \psi(x, z), \psi(x', z') \rangle = (x x') K_z(z, z') + 1,$$

where $K_z(z, z') = \langle \phi(z), \phi(z') \rangle$ is a base kernel defined only over the topic features z . For this assignment, a polynomial kernel of the form

$$K_z(z, z') = (z^\top z' + c)^d$$

was employed, where d and c were tuned via cross-validation.

Interpretation: - The term $x x'$ ensures that the kernel captures the **linear dependence on video length**. - The inner kernel $K_z(z, z')$ models the **nonlinear dependence on topic features**. - The constant $+1$ term introduces a bias dimension. Together, this hybrid kernel unifies the linear and nonlinear effects in a single function, allowing us to train the semi-parametric model using `sklearn.kernel_ridge` in a purely kernelized setting.

Feature Mapping Visualization

$$\text{Original Model: } y = w(z)x + b, \quad w(z) = p^\top \phi(z)$$

$$\Rightarrow y = p^\top (x \phi(z)) + b$$

$$\text{Define augmented feature map: } \psi(x, z) = \begin{bmatrix} x \phi(z) \\ 1 \end{bmatrix}$$

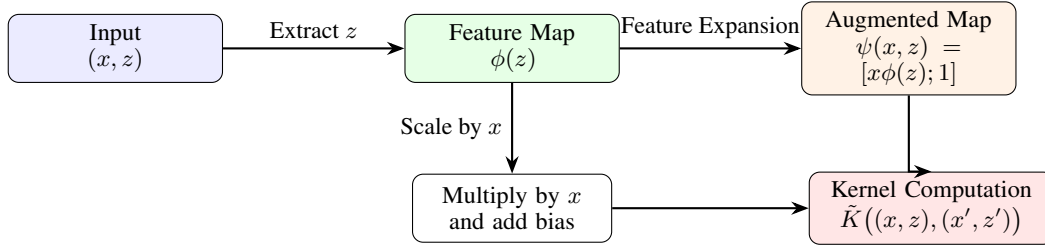
$$\text{Then the model becomes: } y = \tilde{p}^\top \psi(x, z), \quad \tilde{p} = \begin{bmatrix} p \\ b \end{bmatrix}$$

$$\begin{aligned} \text{Hence, the induced kernel is: } \tilde{K}((x, z), (x', z')) &= \langle \psi(x, z), \psi(x', z') \rangle \\ &= (x x') \langle \phi(z), \phi(z') \rangle + 1 \\ &= (x x') K_z(z, z') + 1 \end{aligned}$$

Intuitive Summary:

- $\phi(z)$: maps the topic features z into a nonlinear, high-dimensional space.
- $x \phi(z)$: scales those nonlinear features by video length, linking size and content complexity.
- The $+1$ term adds a bias feature, ensuring flexibility for nonzero intercepts.
- Combined, $\psi(x, z)$ allows linear methods in feature space to model semi-parametric relationships.

Graphical Representation of Feature Transformation



The figure illustrates the flow of information: raw inputs (x, z) are mapped nonlinearly to $\phi(z)$, scaled by x to form the augmented feature map $\psi(x, z)$, and then combined through an inner product to produce the kernel value $\tilde{K}$.

2. Hyperparameter Tuning

We performed a grid search to determine the optimal hyperparameters for the polynomial kernel. We ran it for $d=1,2,3,4,5,6$ and for coefficients $c=0.0, 0.1, 0.2, 0.5, 0.7, 1.0, 2.0, 5.0, 10.0, 100$.

The table shows the results:

Table 1: Average Kernel Time, Training Time, and R^2 for Degrees $d = 1$ to 6 and Coef0 Values

d	c	t_{kernel} (s)	t_{train} (s)	R^2
1	0.0	0.5441	3.0229	0.044350
1	0.1	0.2918	1.4388	0.969698
1	0.2	0.3132	1.6919	0.969698
1	0.5	0.3072	1.7035	0.969697
1	0.7	0.3166	1.7935	0.969697
1	1.0	0.3689	1.4083	0.969697
1	2.0	0.3911	1.7897	0.969697
1	5.0	0.3107	1.6627	0.969697
1	10.0	0.3077	1.6563	0.969697
1	100.0	0.3093	1.4198	0.969697
2	0.0	0.3177	1.7127	0.313882
2	0.1	0.3025	1.6560	0.969930
2	0.2	0.2773	1.6069	0.969927
2	0.5	0.3040	1.5205	0.969926
2	0.7	0.3061	1.6446	0.969926
2	1.0	0.3007	1.7417	0.969926
2	2.0	0.2864	1.4238	0.969926
2	5.0	0.3047	1.6976	0.969926
2	10.0	0.3154	1.7328	0.969926
2	100.0	0.2907	1.6355	0.969926
3	0.0	0.9338	1.6507	0.050794
3	0.1	0.9816	1.7030	0.969976
3	0.2	0.9167	1.6593	0.969944
3	0.5	0.8989	1.6695	0.969939
3	0.7	0.8852	1.7019	0.969939
3	1.0	0.9514	1.6877	0.969938
3	2.0	0.8582	1.7362	0.969938
3	5.0	0.9280	1.7891	0.969938
3	10.0	0.8560	1.7232	0.969938
3	100.0	0.8487	1.7688	0.969938
4	0.0	0.9481	1.6637	0.151858
4	0.1	0.9503	1.6181	0.968927
4	0.2	0.9424	1.6476	0.969839
4	0.5	0.9142	1.6798	0.969775
4	0.7	0.8887	1.7125	0.969774
4	1.0	0.8704	1.5956	0.969773
4	2.0	0.8823	1.4335	0.969773
4	5.0	0.8760	1.5688	0.969773
4	10.0	0.8510	1.6394	0.969773
4	100.0	0.8712	1.6471	0.969762
5	0.0	0.9897	1.5454	0.040165
5	0.1	0.9669	1.5974	0.902361
5	0.2	0.9549	1.5977	0.969720
5	0.5	0.9088	1.6766	0.969540
5	0.7	0.8960	1.7046	0.969536
5	1.0	0.8700	1.6734	0.969536
5	2.0	0.8854	1.7116	0.969536
5	5.0	0.8906	1.7231	0.969536
5	10.0	0.8973	1.6671	0.969536
5	100.0	0.8938	1.6491	0.964892

6	0.0	1.0145	1.5585	0.089972
6	0.1	1.0926	1.6943	0.751965
6	0.2	0.9451	1.7066	0.964866
6	0.5	0.9197	1.7164	0.969548
6	0.7	0.9163	1.6660	0.969544
6	1.0	0.8819	1.7212	0.969543
6	2.0	0.8869	1.6290	0.969543
6	5.0	0.8792	1.5574	0.969543
6	10.0	0.8413	1.6645	0.969543
6	100.0	0.8411	40.1345	-0.876254

From the table above we see that we get that the highest average R^2 score is obtained at

$$d = 3, \quad c = 0.1, \quad R^2 = 0.969976,$$

with an average kernel construction time of 0.9816 s and an average training time of 1.7030 s.

While this configuration achieves the best prediction accuracy, its kernel computation time is noticeably higher compared to lower-degree models. When we also take computational efficiency into account, the configuration

$$d = 2, \quad \text{coef0} = 0.1$$

offers a nearly identical accuracy,

$$R^2 = 0.969930,$$

but with substantially lower computation time: only 0.3025 s for kernel construction and 1.6560 s for model training.

Graphs shown at the end of the report.

Problem 1.2: Delay Recovery by Inverting a XOR Arbiter PUF

4. Inversion Method for the XOR Arbiter PUF Model

A 32-bit arbiter PUF contains 32 stages with delay parameters (a_i, b_i, c_i, d_i) or (p_i, q_i, r_i, s_i) for $0 \leq i \leq 31$. For any arbiter PUF, define

$$\alpha_i = \frac{p_i - q_i + r_i - s_i}{2}, \quad \beta_i = \frac{p_i - q_i - r_i + s_i}{2}.$$

These quantities relate to the linear model of the arbiter PUF as

$$w_0 = \alpha_0, \quad w_i = \alpha_i + \beta_{i-1} \quad (1 \leq i \leq 31), \quad w_{32} = \beta_{31},$$

giving a 33-dimensional model vector $w \in \mathbb{R}^{33}$.

XOR Arbiter Model. Given two arbiter PUFs with model vectors $u, v \in \mathbb{R}^{33}$, the XOR PUF has model

$$w = u \otimes v \in \mathbb{R}^{1089},$$

where $\otimes$ is the Kronecker product. Reshaping w into a 33×33 matrix,

$$W = \text{reshape}(w, 33, 33),$$

we obtain the identity

$$W = uv^\top,$$

i.e. the XOR model is the vectorized outer product of the two arbiter models. Thus W is a rank-1 matrix.

Step 1: Recovering u and v (De-Kroneckering). Since $W = uv^\top$ is rank-1, we compute the SVD(singular value decomposition)

$$W = U\Sigma V^\top.$$

Let σ_1 be the largest singular value and $U_{:,1}, V_{:,1}$ the corresponding singular vectors. Then

$$\hat{u} = \sqrt{\sigma_1} U_{:,1}, \quad \hat{v} = \sqrt{\sigma_1} V_{:,1}.$$

Any scaling $c \neq 0$ preserves the model because $\frac{1}{c}\hat{u} \otimes (c\hat{v}) = \hat{u} \otimes \hat{v}$. Hence $\hat{u}, \hat{v}$ serve as valid arbiter model vectors.

Step 2: Recovering α_i, β_i from the arbiter model. Given any recovered arbiter model vector $w^{(\text{arb})} \in \mathbb{R}^{33}$, we must find α_i, β_i satisfying the linear relations. This system is underdetermined, so we choose a consistent constructive solution:

$$\beta_0 = \beta_1 = \dots = \beta_{30} = 0, \quad \beta_{31} = w_{32}^{(\text{arb})},$$

$$\alpha_0 = w_0^{(\text{arb})}, \quad \alpha_i = w_i^{(\text{arb})} \quad \text{for } 1 \leq i \leq 31.$$

Substituting verifies that all model equations are satisfied.

Step 3: Recovering delays from α_i, β_i . From the definitions of α_i, β_i , set

$$D_i^{(1)} = p_i - q_i = \alpha_i + \beta_i, \quad D_i^{(2)} = r_i - s_i = \alpha_i - \beta_i.$$

Because delays must be non-negative but their magnitudes are unconstrained, we construct a valid solution as follows:

$$(p_i, q_i) = \begin{cases} (D_i^{(1)}, 0) & D_i^{(1)} \geq 0, \\ (0, -D_i^{(1)}) & D_i^{(1)} < 0, \end{cases}$$

$$(r_i, s_i) = \begin{cases} (D_i^{(2)}, 0) & D_i^{(2)} \geq 0, \\ (0, -D_i^{(2)}) & D_i^{(2)} < 0. \end{cases}$$

This ensures $p_i, q_i, r_i, s_i \geq 0$ and exactly reproduces the required α_i, β_i . Applying this procedure to both $\hat{u}$ and $\hat{v}$ yields 256 non-negative delays that regenerate the original XOR model.

Summary. The inversion method consists of:

1. Reshaping the XOR model into a 33×33 matrix.
2. Extracting two arbiter models via the rank-1 SVD factorization.
3. Converting each arbiter model into α_i, β_i .
4. Constructing one valid non-negative set of delays satisfying the equations.

Graphs for 1.1 part 2: Hyperparameter tuning

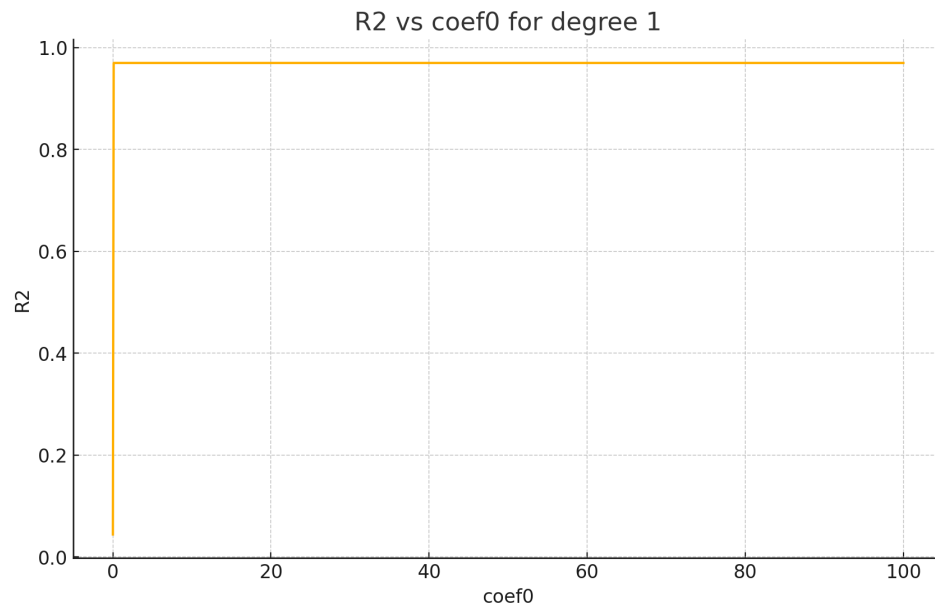


Figure 1: R² vs coef0 for degree 1

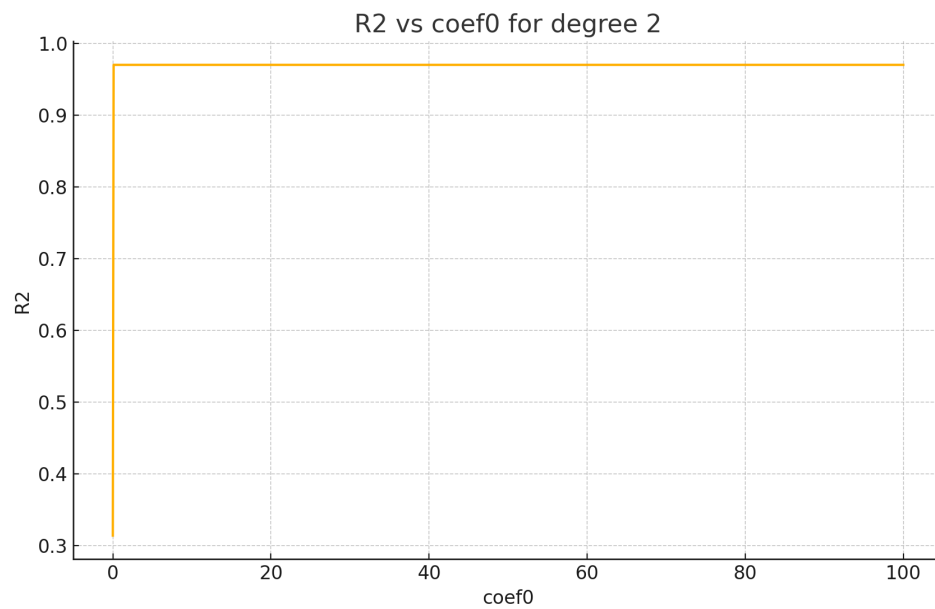


Figure 2: R² vs coef0 for degree 2

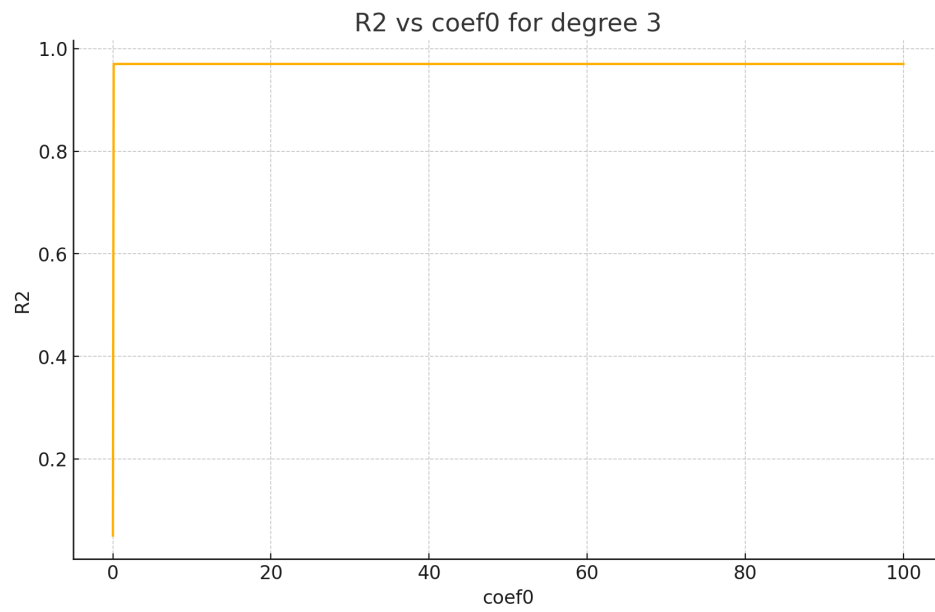


Figure 3: R^2 vs coef0 for degree 3

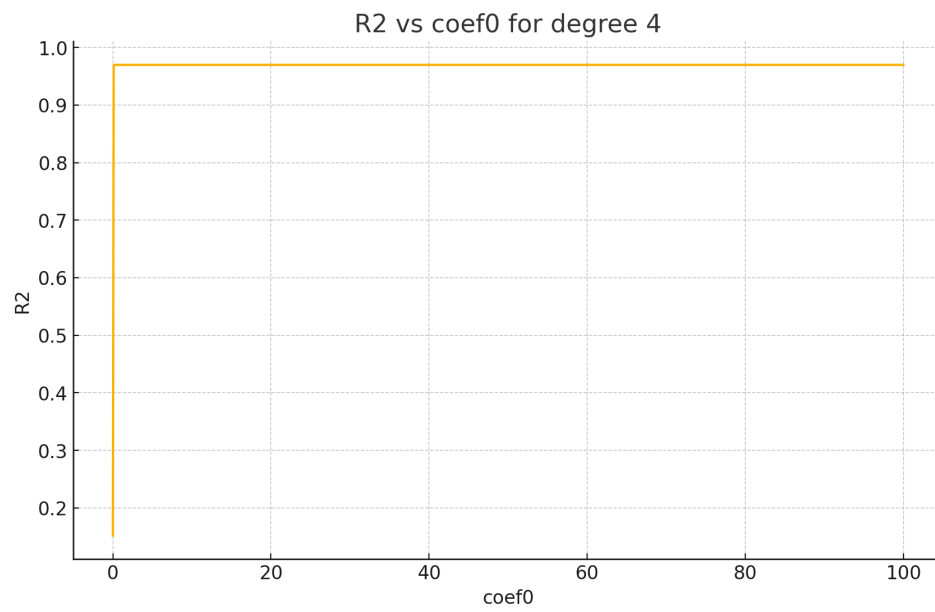


Figure 4: R^2 vs coef0 for degree 4

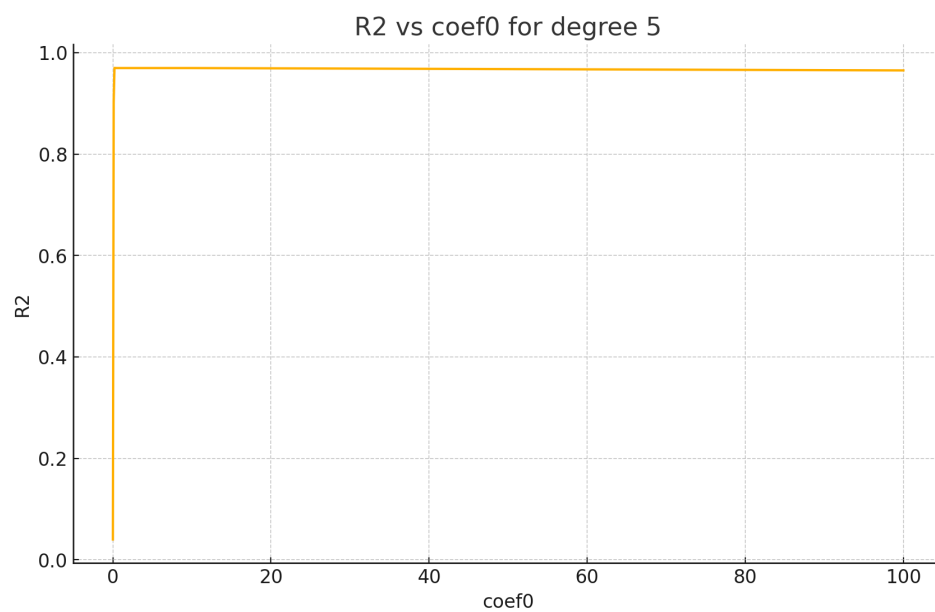


Figure 5: R^2 vs coef0 for degree 5

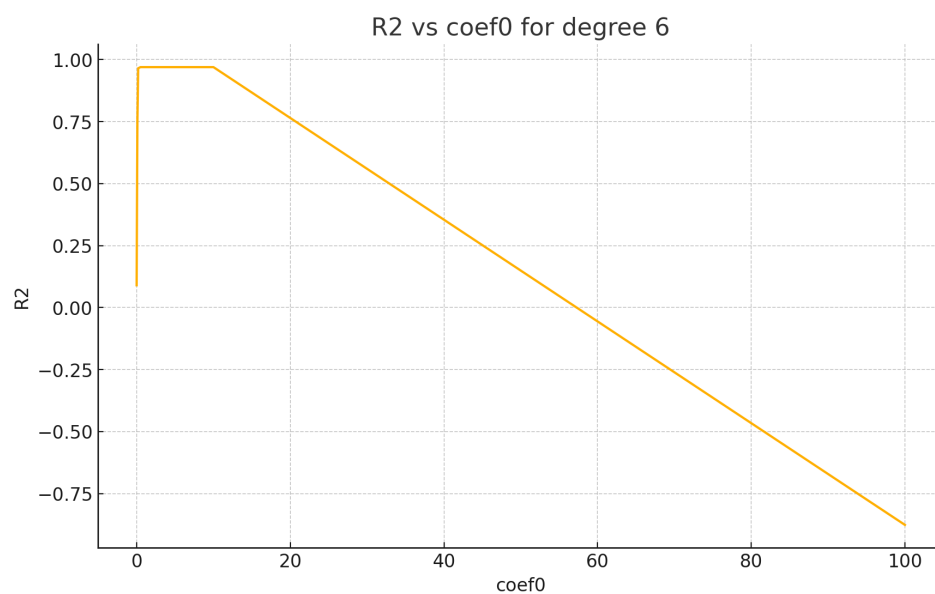


Figure 6: R^2 vs coef0 for degree 6

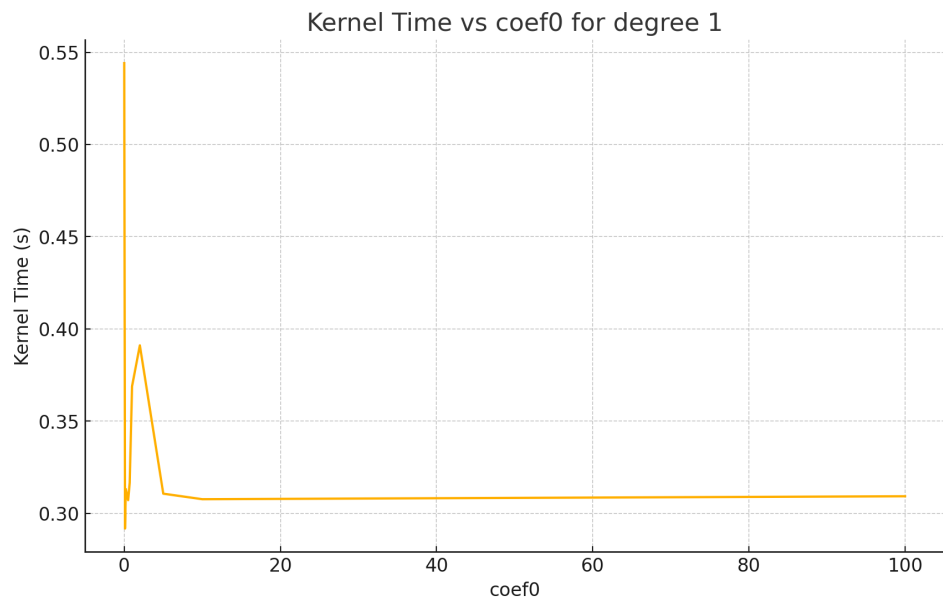


Figure 7: Kernel Time vs coef0 for degree 1

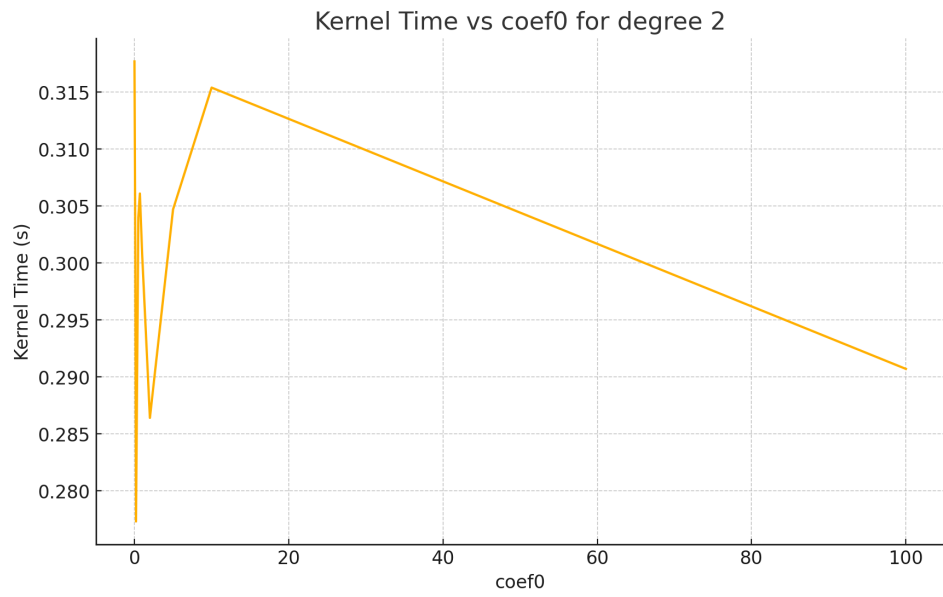


Figure 8: Kernel Time vs coef0 for degree 2

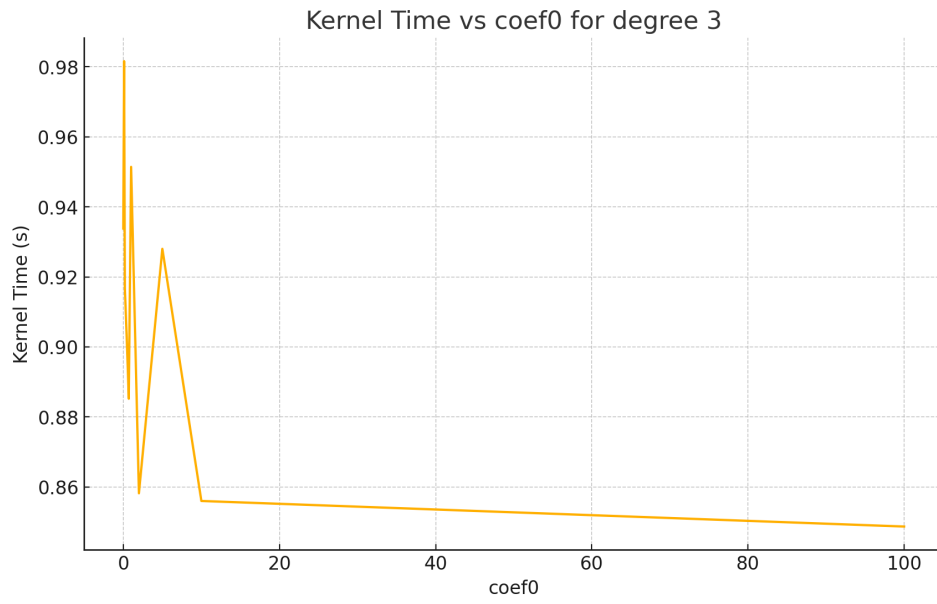


Figure 9: Kernel Time vs coef0 for degree 3

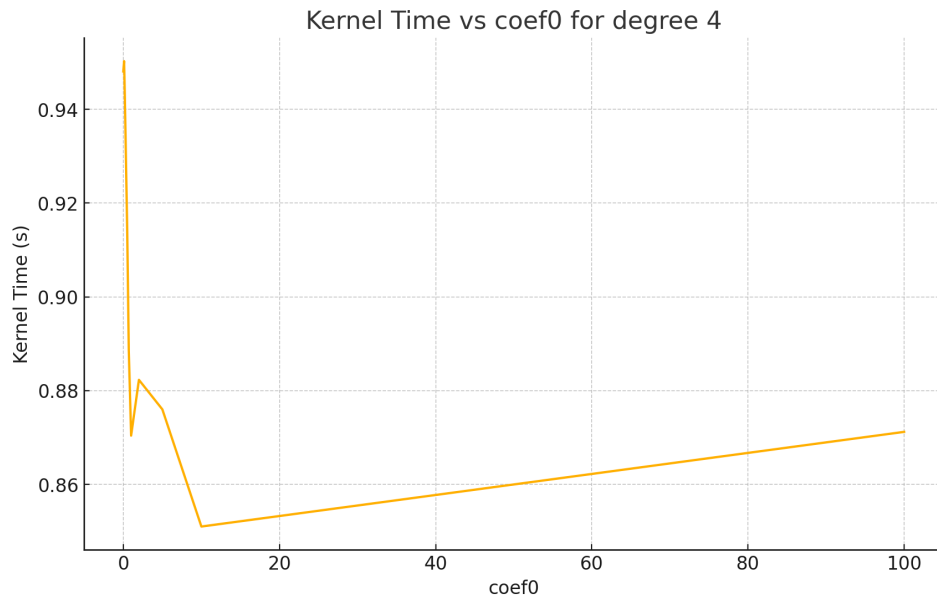


Figure 10: Kernel Time vs coef0 for degree 4

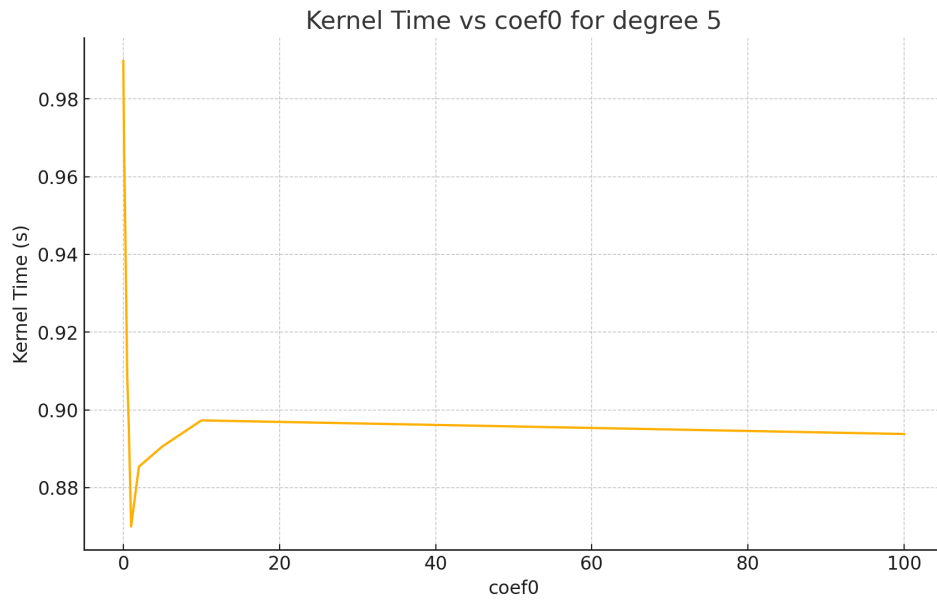


Figure 11: Kernel Time vs coef0 for degree 5

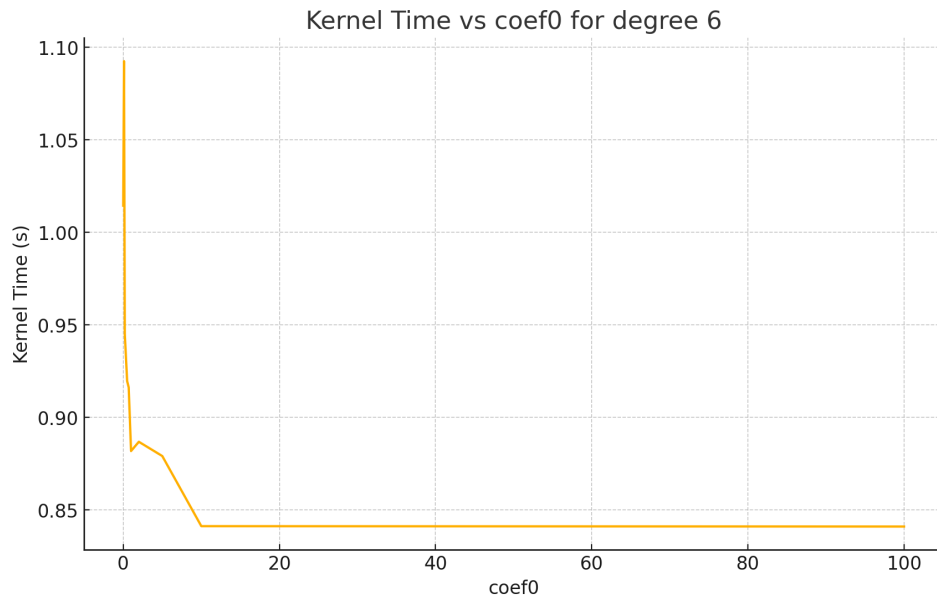


Figure 12: Kernel Time vs coef0 for degree 6

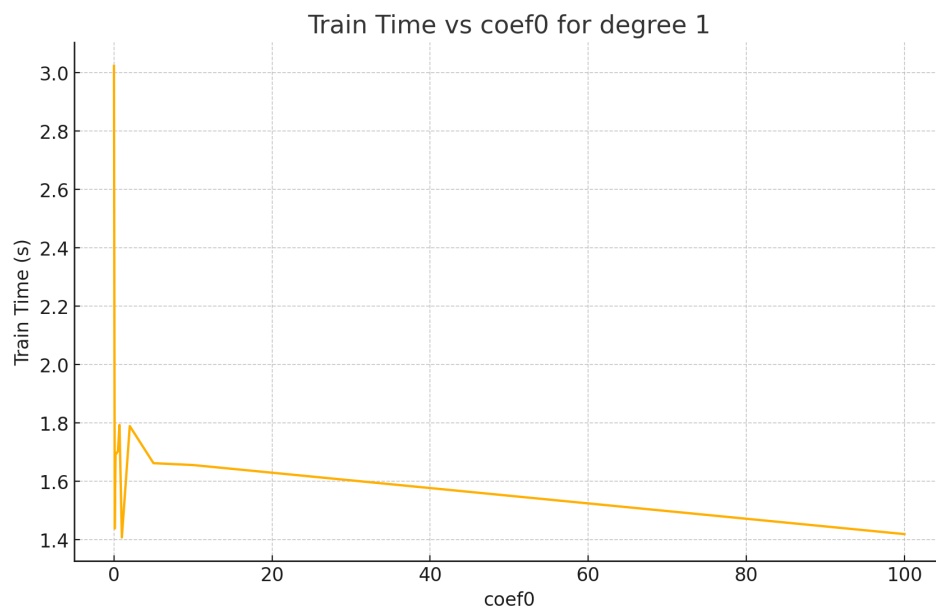


Figure 13: Train Time vs coef0 for degree 1

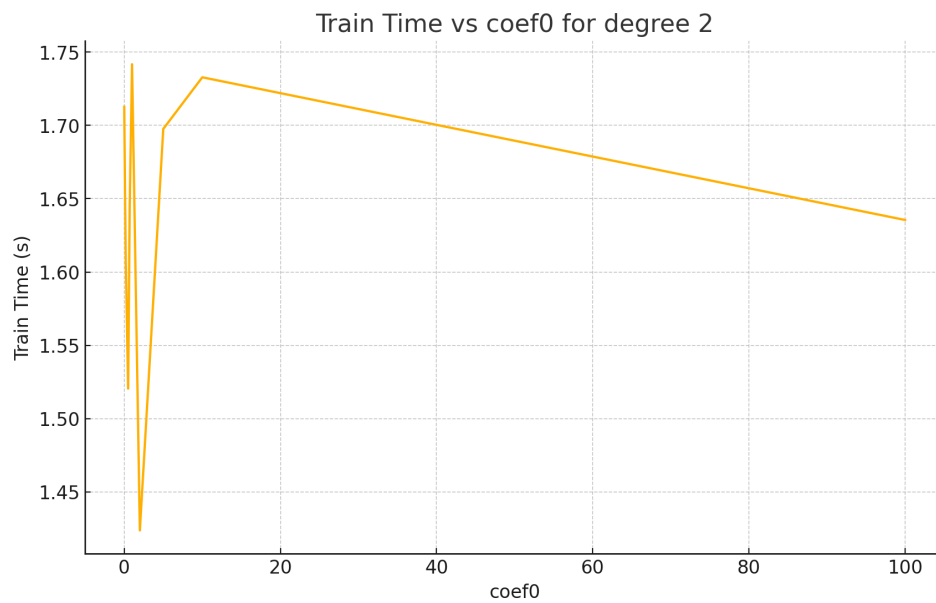


Figure 14: Train Time vs coef0 for degree 2

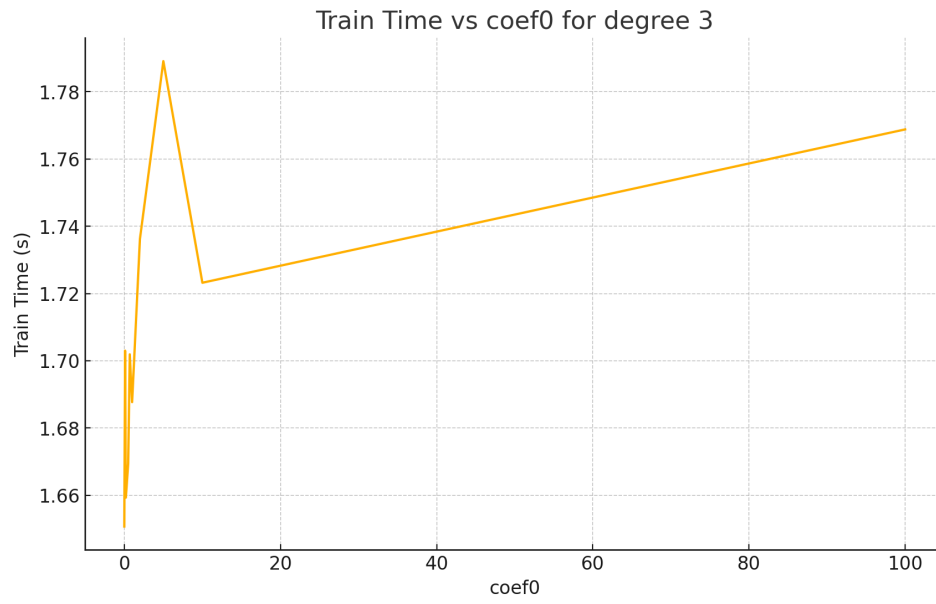


Figure 15: Train Time vs coef0 for degree 3

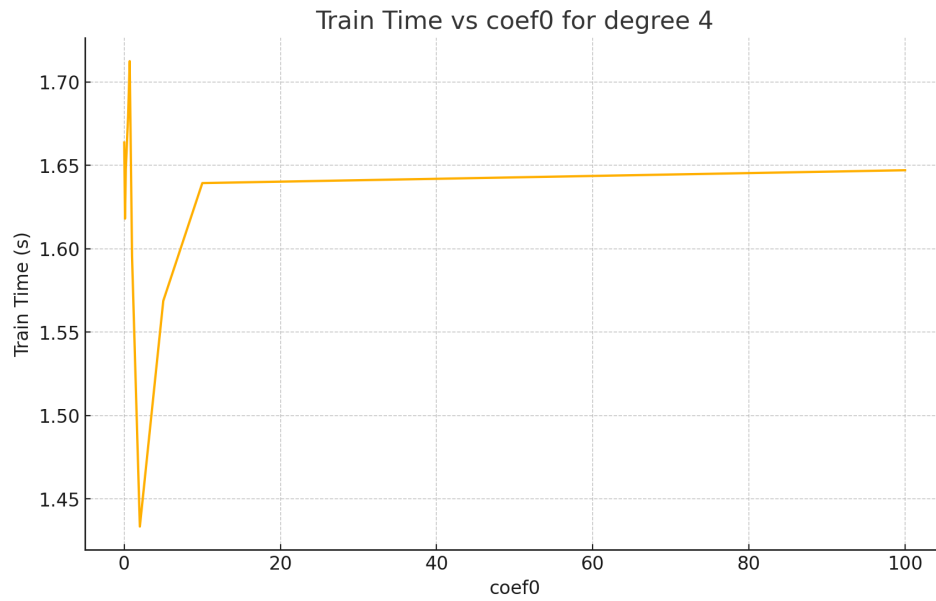


Figure 16: Train Time vs coef0 for degree 4

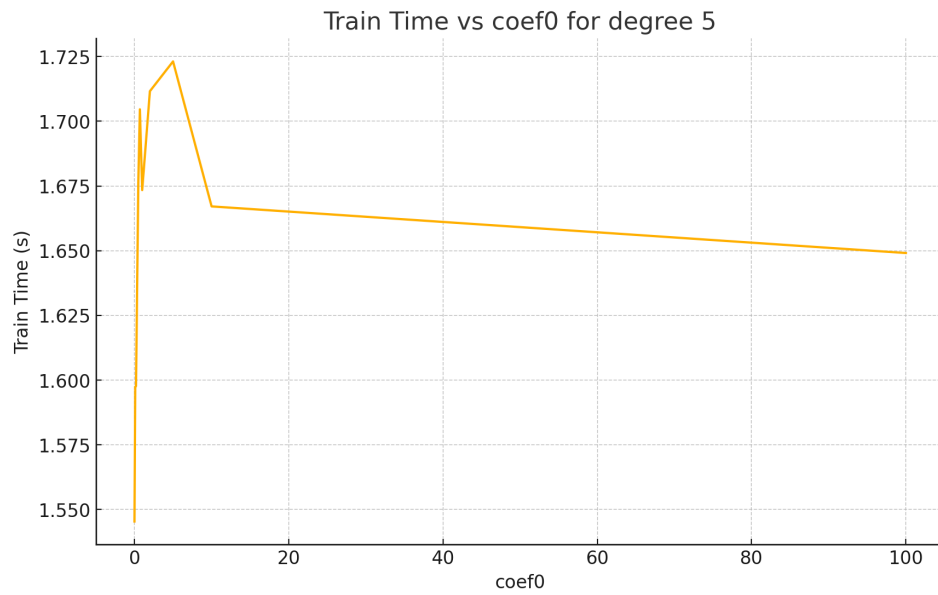


Figure 17: Train Time vs coef0 for degree 5

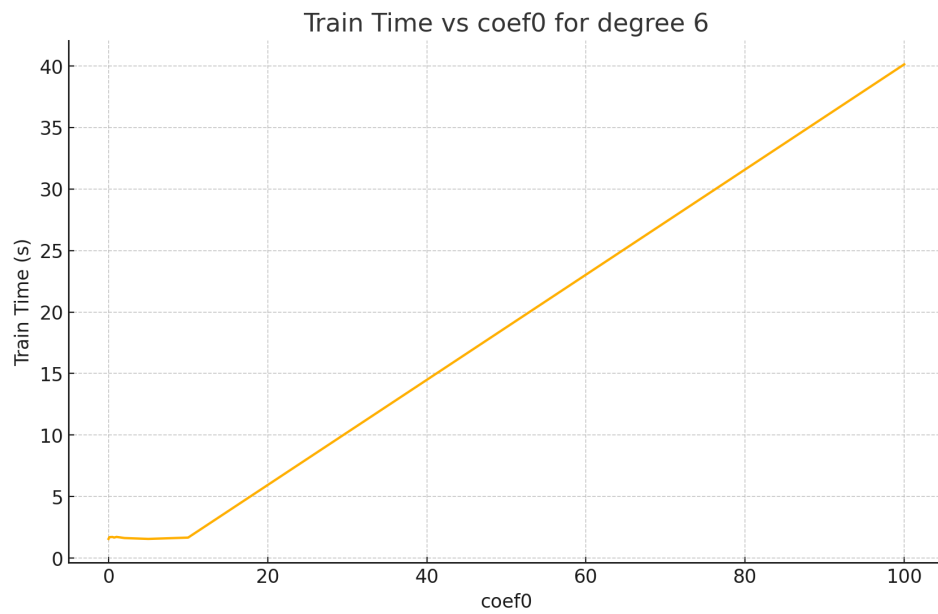


Figure 18: Train Time vs coef0 for degree 6